

theFlow!

Technical Architecture Document

Version 0.1.0 | June 2026

Author: Xavier Garès | License: GPLv3

1. Overview

theFlow! is a cross-platform, freeform visual canvas application that allows users to place, connect, and organise heterogeneous media nodes — images, video, audio, documents, and text — in an infinite 2D workspace. Nodes are interconnected with Bézier curves, grouped with Backdrops, and annotated with Sticky Notes and canvas paint strokes.

The application is built entirely in Python using the PyQt6 framework, serialises workspaces to a custom JSON-based .flow file format, and is distributed as self-contained native binaries on macOS, Windows, and Linux (DEB, RPM, and AppImage).

1.1 Key Capabilities

- Infinite freeform canvas with smooth zoom and pan
- 7 node types: Text, Image, Movie, Audio, Document, Paint, Dot
- Inline media viewers anchored below nodes (video, audio waveform, PDF viewer)
- Canvas annotation (freehand drawing overlaid on the canvas)
- Bézier connection curves between nodes with socket orientation
- Backdrop grouping with child pinning and recursive refresh
- Full undo/redo stack with per-operation snapshots
- Import/export of node subsets (.flow files)
- Light and dark themes with live settings propagation
- Cross-platform builds: macOS (.app + DMG), Windows (.exe + Inno Setup), Linux (.deb, .rpm, .AppImage)
- File association: .flow files open theFlow! on double-click on all platforms

1.2 Target Users

User Type	Primary Use
Creative professionals	Mood boards, visual research, project planning
Educators	Visual lesson planning, resource organisation
Researchers	Reference management, visual note-taking
Product managers	Flow diagrams, feature mapping with media references
Developers	Architecture diagrams with embedded documentation

2. Technology Stack

2.1 Core Language & Runtime

Component	Version	Role
Python	3.9–3.12	Primary language (3.9 pinned for Linux; 3.12 for macOS/Windows)
PyQt6	6.4.2+ (Linux/Win), latest (macOS)	UI framework — Qt6 bindings for Python
PyQt6-sip	≥13.4	C++ ↔ Python binding layer
PyQt6-Qt6	matches PyQt6	Qt6 shared libraries

2.2 Qt6 Modules Used

Qt Module	Purpose
QtWidgets	QGraphicsScene, QGraphicsView, QDialog, QMainWindow
QtCore	QTimer, QPointF, QRectF, QSizeF, QTimeLine, QEvent
QtGui	QPainter, QColor, QPen, QBrush, QPixmap, QFont, QSvgRenderer
QtMultimedia	QMediaPlayer, QAudioOutput, QAudioDecoder (audio/video)
QtMultimediaWidgets	QVideoWidget (video inline viewer)
QtSvg / QtSvgWidgets	SVG logo overlay and node badges
QtNetwork	Network access
QtPrintSupport	Print support
QtOpenGL	OpenGL context for multimedia compositing
QtPdf / QtPdfWidgets	PDF inline viewer

2.3 Third-Party Dependencies

Package	Purpose	Platform
PyInstaller 6.x	Freezes Python app into standalone binary	All
Pillow	Icon format conversion (PNG → ICO/ICNS)	Build time
cairosvg	SVG → PNG conversion for icons	Build time
pydub	Audio waveform generation (RMS analysis)	All
dpkg-dev	Builds .deb packages	Linux DEB
rpmbuild	Builds .rpm packages	Linux RPM
appimagetool	Builds portable AppImage	Linux AppImage
Inno Setup 6	Builds Windows .exe installer	Windows

3. Application Architecture

3.1 Module Structure

theFlow! is structured as a collection of flat Python modules — no package hierarchy. All modules live in the project root directory.

Module	Responsibility
main.py	Entry point. QApplication subclass, settings loading, file-open event handling
main_linux.py	Linux entry point — sets QT_PLUGIN_PATH, LD_LIBRARY_PATH before QApplication
view_logic.py	QGraphicsView subclass. Pan/zoom, keyboard shortcuts, context menu, node creation, file I/O
scene_logic.py	QGraphicsScene subclass. Serialisation (to_dict/from_dict), undo/redo stack, backdrop containment
node.py	All node types: Node, ImageNode, MovieNode, AudioNode, DocumentNode, PaintNode, Dot — plus inline viewers
curve.py	ConnectionLine (Bézier), Socket
backdrop.py	Backdrop with child pinning and recursive curve refresh
note.py	StickyNote, ResizeHandle
paint.py	PaintNode with QGraphicsPixmapItem canvas
paint_on_canvas.py	CanvasPainter, CanvasStroke, StrokeToolbar (freehand annotation)
menu.py	Right-click context menu builder

ui_components.py	Tab creation menu (QMenu)
settings.py	SettingsDialog with tabbed configuration panels
config.py	Global constants, file paths, colour palettes, platform detection
utils.py	Shared helpers: bezier math, RichTextEditDialog, menu styling, drag state
logo.py	LogoOverlay — animated SVG fade on empty canvas
rthook.py / rthook_linux.py	PyInstaller runtime hooks — fix Qt plugin paths and exception handling
hook_fix_cfbundle.py	macOS runtime hook — fixes CFBundle reference and Qt fatal message handler

3.2 Application Entry Flow

On launch, main.py executes the following sequence:

- Set Qt environment variables (audio backend, OpenGL mode for frozen binary)
- Instantiate TheFlowApp (QApplication subclass with QFileOpenEvent support)
- Load or create settings.json from the platform-appropriate config directory
- Import View (QGraphicsView subclass) — deferred to avoid Qt init issues
- Construct View, apply settings, show window
- If sys.argv[1] exists or a QFileOpenEvent was received: open the .flow file after 400ms delay
- Enter Qt event loop

3.3 Settings System

Platform	Settings Path
macOS (frozen)	Next to theFlow.app/Contents/MacOS/theFlow
macOS (dev)	Project root /settings/settings.json
Windows (frozen)	Next to theFlow.exe
Linux (frozen — DEB/RPM/AppImage)	~/.config/theflow/settings.json (XDG)
Linux (dev)	Project root /settings/settings.json

On startup, settings are loaded and merged with DEFAULTS. Settings are applied live — changing a setting propagates immediately to all canvas items via update_from_settings().

4. Canvas & Scene System

4.1 QGraphicsScene / QGraphicsView

The canvas is built on Qt's QGraphicsScene + QGraphicsView model:

- Scene (scene_logic.py / FlowScene): holds all items, manages serialisation and undo stack
- View (view_logic.py / View): handles viewport transformation, input events, drag-and-drop

The scene uses an infinite coordinate system. The view applies smooth scaling (Ctrl+scroll or pinch) and panning (middle mouse or space+drag).

4.2 Coordinate System

All node positions are stored in scene coordinates. Inline viewers (media players, PDF views) are parented to the viewport widget — not the scene — and their pixel position is recalculated on every scroll, zoom, or node move via `reposition()`. This avoids Qt's limitation that QWidget children of QGraphicsItems don't scale with the scene transform.

4.3 Z-Order

Layer	Z-Value	Items
Background	0	Backdrop
Nodes	10	All node types, Dot
Sticky Notes	15	StickyNote
Curves	5	ConnectionLine
Handles	30	ResizeHandle, BackdropResizeHandle
Temp line	100	Dragging connection preview
Annotation strokes	200	CanvasStroke

4.4 Undo / Redo

The undo stack stores operation dicts. Supported operation types:

- add / delete — node/item creation and removal
- move — position change
- resize — size change for Backdrop and StickyNote
- connect / disconnect — curve creation and removal
- backdrop_label / backdrop_color / backdrop_font_size / backdrop_font_color
- sticky_text / sticky_color / sticky_font_size / sticky_font_color
- node_label / node_color / node_font_size / node_font_color / node_shape

- `add_stroke / del_stroke / clear_strokes` — canvas annotation
- `doc_node_edit` — media node path changes

5. Node System

5.1 Base Node

All content nodes inherit from `Node (QGraphicsItem)`. The base class provides:

- Title (`NodeTitle` — `QGraphicsTextItem`): editable label shown on node face
- Input socket (`Socket` — `QGraphicsEllipseItem`): left or top, depending on orientation
- Output socket (`Socket`): right or bottom
- Shape: rectangle, rounded_rectangle, ellipse, diamond, hexagon
- Paint method: selection glow (blue, 10-pass radial gradient), node fill, border, optional icon badge
- Context menu: color, font size, font color, shape, orientation, copy/cut/paste
- Double-click: opens `RichTextEditDialog` (name + rich text body)

5.2 Node Types

Type	Key	Feature	Inline Viewer
Text Node	T	Rich text with formatting	QTextBrowser with hyperlinks
Image Node	I	Loads PNG/JPG/GIF/WebP/SVG	Scaled image with zoom grip
Movie Node	M	Loads MP4/MOV/AVI/MKV	QVideoWidget + click-anywhere timeline
Audio Node	A	Loads MP3/WAV/AAC/FLAC	RMS waveform + playhead seek
Document Node	D	Loads PDF (via <code>QPdfDocument</code>)	QPdfView — scrollable, FitToWidth
Paint Node	P	Embedded pixel canvas (<code>QPixmap</code>)	Resizable paint canvas with brushes
Dot	Q	Connection pass-through point	None

5.3 Socket & Connection System

Sockets are `QGraphicsEllipseItem` children of each node. Connections are drawn as `QGraphicsPathItem (ConnectionLine)` using cubic Bézier curves. Control point direction follows socket orientation (left-right, right-left, top-bottom, bottom-top). Multiple outputs can connect to one input; one output can connect to multiple inputs.

5.4 Inline Viewer Architecture

Viewer Class	Node	Widget Used
InlineTextViewer	Text Node	QTextBrowser (read-only, hyperlinks)
InlineImageViewer	Image Node	QLabel + QScrollArea + resize grip
InlineVideoPlayer	Movie Node	QVideoWidget + _ClickSlider timeline
InlineAudioPlayer	Audio Node	SimpleWaveformWidget + _ClickSlider
InlineDocViewer	Document Node	QPdfView (FitToWidth, scrollable)

6. Audio & Video Pipeline

6.1 Audio Node

Audio playback uses Qt's QMediaPlayer + QAudioOutput. The waveform display uses SimpleWaveformWidget:

- WAV files: synchronous decode via Python wave module
- MP3/AAC/FLAC: asynchronous decode via QAudioDecoder — buffers accumulated and RMS computed per-frame
- Waveform rendered as smooth tapered silhouette with vertical gradient (teal → darker teal)
- Playhead: vertical line updated on QMediaPlayer.positionChanged
- Click/drag to seek: custom _ClickSlider maps click x-position to media duration

6.2 Video Node

Video playback uses QMediaPlayer + QVideoWidget. The same _ClickSlider timeline provides click-anywhere seeking. The inline viewer shows video at native aspect ratio inside a fixed container.

6.3 FFmpeg Backend

Qt6 Multimedia uses the FFmpeg backend on all platforms (bundled inside the PyInstaller binary). GStreamer is NOT used. UPX compression is disabled on Linux builds because it corrupts the bundled FFmpeg shared libraries. The ApplImage also bundles the ffmpeg binary separately for pydub audio waveform analysis.

7. Serialisation — The .flow File Format

7.1 Overview

A .flow file is a UTF-8 JSON file — human-readable and version-independent. The root object contains:

- nodes — array of node snapshot dicts
- connections — array of {from_socket, to_socket} string pairs
- strokes — array of canvas annotation stroke dicts (color, width, style, points)

7.2 Common Node Fields

Field	Type	Description
id	int	Python id() of the item — used as key for socket mapping
type	string	Node type identifier
name	string	Display label
x, y	float	Scene position
shape	string	rectangle / rounded_rectangle / ellipse / diamond / hexagon
color	string	ARGB hex color (#AARRGGBB)
color_locked	bool	Whether color was manually set
font_size	int	Canvas font size
font_color	string	ARGB hex
orientation	string	left-right / right-left / top-bottom / bottom-top

7.3 Node Type Identifiers

Type string	Node class	Extra fields
node	Node (text)	title_html, body_html
image	ImageNode	image_path, image_data (base64 PNG)
movie	MovieNode	movie_path
audio	AudioNode	audio_path, color (as [r,g,b,a])
doc	DocumentNode	doc_path
paint	PaintNode	canvas_data (base64 PNG)
dot	Dot	—
sticky	StickyNote	sticky_name, sticky_text, color, font_size,

		font_color, width, height
backdrop	Backdrop	backdrop_name, backdrop_text, color, font_size, font_color, width, height

7.4 Connections

Connections are serialised as string pairs: { "from": "123456789_out", "to": "987654321_in" }. The socket ID format is {node_id}_{in|out}. During from_dict(), a socket_map dict resolves these strings back to live Socket objects.

7.5 Import / Export

Export Selected (Ctrl+E) serialises only selected items and their interconnecting curves. Import Nodes (Ctrl+I) calls merge_from_dict() — additive load that does not clear the scene. Imported nodes are offset by 80px to avoid exact overlap.

8. Canvas Annotation

Canvas annotation (Ctrl+P) overlays freehand drawing on the QGraphicsScene using CanvasStroke items (QGraphicsPathItem subclass). Strokes are serialised in the .flow file as arrays of QPointF coordinates with color, width, and style (solid, dashed, dotted). The StrokeToolbar is a floating QWidget providing color picker, stroke thickness slider, line style selector, and clear-all button.

9. Build & Distribution

9.1 Overview

Platform	Command	Output
macOS	./build_mac.sh	theFlow.app + theFlow-0.1.0.dmg
Windows	build_windows.bat	theFlow.exe + theFlow-0.1.0-setup.exe
Linux DEB	./build_deb.sh	theflow + theflow_0.1.0_amd64.deb
Linux RPM	./build_rpm.sh	theflow + theflow-0.1.0-1.x86_64.rpm
Linux AppImage	./build_appimage.sh	theflow + theFlow-0.1.0-x86_64.AppImage

9.2 macOS Build

File	Purpose
build_mac.sh	Build script — creates venv, installs deps, converts icon, runs

	PyInstaller, code-signs, creates DMG
theflow_mac.spec	PyInstaller spec — datas, hiddenimports, info_plist with CFBundleDocumentTypes
hook_fix_cfbundle.py	Runtime hook — fixes CFBundle reference, installs Qt message handler
rthook_theflow.py	Runtime hook — sets sys.path and Qt plugin paths from _MEIPASS
main.py	Entry point — TheFlowApp subclass with QFileOpenEvent support

9.3 Windows Build

File	Purpose
build_windows.bat	Build script — runs PyInstaller then Inno Setup
theflow_windows.spec	PyInstaller spec — onefile mode, hiddenimports, version_info.txt
theflow_setup.iss	Inno Setup script — installs exe, registers .flow in HKCU registry
main_windows.py	Entry point — sys.argv file open with 400ms delay

9.4 Linux DEB Build

File	Purpose
build_deb.sh	Runs PyInstaller, assembles package tree, runs dpkg-deb
theflow_linux.spec	PyInstaller spec for Linux — FFmpeg bundled, no macOS hooks
setup_ubuntu.sh	First-time setup: installs Python 3.9, PyQt6 6.4.2
theflow.desktop	FreeDesktop .desktop entry with MimeType=application/x-theflow
theflow-mime.xml	MIME type declaration for .flow extension
main_linux.py	Entry point — sets QT_PLUGIN_PATH, LD_LIBRARY_PATH from _MEIPASS

9.5 Linux RPM Build

The RPM build follows the same PyInstaller step but packages via rpmbuild. build_rpm.sh creates the ~/rpmbuild tree, tarballs dist/theflow, copies assets to SOURCES/, copies theflow.spec to SPECS/, runs rpmbuild -bb, and copies the .rpm to dist/.

9.6 Linux AppImage Build

The AppImage is a portable self-contained single-file format built by build_appimage.sh:

- Runs on any x86_64 Linux with glibc 2.31+ (Ubuntu 20.04+, Fedora 33+, Debian 11+)

- Build on Ubuntu 20.04 LTS for maximum compatibility
- Bundles: Python runtime, PyQt6, Qt6, FFmpeg libs, ffmpeg binary (for pydub waveforms)
- No installation required — `chmod +x` and run
- Settings stored at `~/.config/theflow/settings.json` (XDG standard)
- Limitation: MIME type and file association not registered automatically

10. File Association

Platform	Mechanism	Handler
macOS	CFBundleDocumentTypes + UTExportedTypeDeclarations in Info.plist	QFileOpenEvent → TheFlowApp.event() → view.open_path()
Windows	HKCU\Software\Classes\.flow registry key (written by Inno Setup)	sys.argv[1] → view.load_file() with 400ms delay
Linux DEB/RPM	MIME type (application/x-theflow) via theflow-mime.xml + .desktop	sys.argv[1] → view.load_file() with 400ms delay
Linux AppImage	Manual setup required — MIME not registered automatically	sys.argv[1] → view.load_file() with 400ms delay

11. Platform Differences & Known Constraints

Feature	macOS	Windows	Linux
Python version	3.12 (venv)	3.12 (system)	3.9 (system)
PyQt6 version	Latest	Latest	6.4.2 (pinned)
Exit method	os._exit()	sys.exit()	sys.exit()
Audio backend	AVFoundation (coreaudio)	Windows Media	FFmpeg (bundled)
UPX compression	✓ Allowed	✓ Allowed	✗ Disabled (corrupts FFmpeg libs)
Icon format	.icns (iconutil)	.ico (Pillow)	.png (256x256)
Settings location	Next to exe / dev root	Next to exe	~/.config/theflow/
Installer	DMG (drag to Applications)	.exe (Inno Setup)	.deb, .rpm, or .AppImage

12. Key Design Decisions

12.1 Flat Module Structure

All modules are flat Python files rather than a package. This simplifies PyInstaller bundling and keeps import paths short.

12.2 Viewport-Parented Inline Viewers

Inline viewers are QWidget instances parented to the viewport, not to scene items. This avoids severe Qt limitations: QWidget children of QGraphicsProxyWidget don't support multimedia (QVideoWidget, QPdfView) and have clipping issues. The manual reposition() pattern is reliable across all Qt multimedia widget types.

12.3 JSON Serialisation

The .flow format uses plain JSON rather than a binary format or SQLite. Files are human-readable, diffable in version control, and forward-compatible. Base64-encoded PNG is used for embedded binary data.

12.4 Separate Entry Points per Platform

macOS, Windows, and Linux each have a separate entry point (main.py / main_linux.py). This avoids a single entry point cluttered with platform guards and makes each platform's startup sequence easy to read and debug independently.

13. Dependency Installation Reference

macOS (one-time)

```
brew install python@3.12  
./build_mac.sh # handles all other deps in venv
```

Windows (one-time)

```
pip install pyinstaller PyQt6 PyQt6-Qt6 PyQt6-sip  
winget install JRSSoftware.InnoSetup  
python convert_icon_windows.py # create icons/logo.ico  
build_windows.bat
```

Linux Ubuntu/Debian (one-time)

```
chmod +x setup_ubuntu.sh && ./setup_ubuntu.sh  
./build_deb.sh # or ./build_appimage.sh
```

Linux Fedora/RHEL (one-time)

```
sudo dnf install python3 python3-devel python3-pip rpm-build rpmdevtools
```

```
pip3 install pyinstaller 'PyQt6==6.4.2' 'PyQt6-Qt6==6.4.2'  
./build_rpm.sh
```

14. Glossary

Term	Definition
.flow file	theFlow! workspace file — JSON containing nodes, connections, and annotation strokes
Backdrop	A translucent grouping rectangle that pins child nodes and moves them together
CanvasStroke	A freehand annotation path drawn over the canvas (QGraphicsPathItem)
ConnectionLine	A Bézier curve connecting two node sockets
DXA	Document XML Attributes unit — 1/1440th of an inch, used by Word/docx
FFmpeg backend	Qt6 Multimedia's media playback engine (replaces GStreamer on Linux in Qt6)
FitToWidth	QPdfView zoom mode — scales each PDF page to fill the viewer width
Inline viewer	A QWidget anchored below a node showing media content (video, audio, PDF)
_MEIPASS	PyInstaller temp directory where frozen binary extracts bundled files at runtime
Node	A rectangular canvas item with an input and output socket
QFileOpenEvent	Qt event sent to the application when macOS asks it to open a file
RMS	Root Mean Square — amplitude measure used to compute the audio waveform silhouette
Socket	A circular connection point on a node (input or output)
StrokeToolbar	Floating toolbar shown during canvas annotation mode
UTI	Uniform Type Identifier — macOS system for declaring file types (com.xavigares.theflow.document)
venv	Python virtual environment — isolated Python + packages used for macOS builds